CS 422 Assignment 4
Part 2: Arbitrary Traffic Matrix

Kush, Joseph, Issac

Main idea

In Part 1 every pair sent the same amount, so the best I could do was make every node close to every other node. Here the traffic matrix T is given, and some pairs send a lot more than others. So the topology should depend on T: pairs with a lot of traffic should get direct links (even more than one link), and pairs with little traffic can share capacity over 2 or more hops.

Choosing the links and routing the flow depend on each other, so I put both into one integer program. For each given T, I solve it, and the solution gives the links to build. Since the solver looks at every possible topology and routing and returns the best one, the topology it gives is the one that maximizes concurrent flow for that T. Here $n = 8$ and $d = 4$.

Formulation

Decision variables.

- $x_{uv} \in \{0, 1, \ldots, 4\}$ (integer), for $u \neq v$: number of links from u to v. More than one link between the same pair is allowed.
- $f^s_{uv} \geq 0$: amount of flow from source s that goes over link $u \to v$.
- $\theta \geq 0$: the fraction of every demand that gets delivered.

Objective.

$$\max \ \theta$$

Constraints.

1. **Degree:** every node has 4 links out and 4 links in.

$$\sum_{v \neq u} x_{uv} = 4, \qquad \sum_{v \neq u} x_{vu} = 4 \qquad \forall u$$

2. **Flow conservation:** source s sends $\theta \cdot T_{st}$ to every destination t.

$$\sum_v f^s_{sv} - \sum_v f^s_{vs} = \theta \sum_t T_{st} \qquad \forall s$$

$$\sum_v f^s_{vw} - \sum_v f^s_{wv} = \theta \, T_{sw} \qquad \forall s, \ \forall w \neq s$$

3. **Capacity:** each link has capacity 1, so the total flow on $u \to v$ is at most the number of links there.

$$\sum_s f_{uv}^s \le x_{uv} \qquad \forall u \ne v$$

I group flow by source instead of by (source, destination) pair. The flow from s only needs to drop off θT_{st} at each t, so this is enough and uses fewer variables.

Why this works

- The degree constraints make sure x is a valid topology (4 links in and out per node).
- The flow and capacity constraints make sure the traffic θT can actually be routed on that topology.
- The solver maximizes θ over all valid topologies and routings together, so the answer is the best topology for this T. It adapts to T because T shows up in the flow conservation constraints, so a different T gives different links.
- The intuition shows up in the results: big entries of T get direct links, and if T is something like $4\times$ a permutation, the solver just builds those links and gets $\theta = 1$.

Implementation (Gurobi)

Code: `https://github.com/krash3125/cs422-assignment-4/blob/main/topology.py`

- `best_topology(T)`: builds and solves the integer program and returns θ and the links.
- Variables x, f, θ; objective; degree constraints; flow conservation; capacity.
- `random_hose_tm()`: makes a random hose traffic matrix (a random mix of permutations, scaled by 4) for testing.

Results

traffic matrix	best θ
uniform ($\frac{4}{7}$ to every pair)	0.700
random hose #0	0.875
random hose #1	0.861
random hose #2	0.827

For the uniform matrix the solver gets 0.700. This matches Part 1: with $d = 4$, each node can reach 4 nodes in 1 hop and the other 3 in 2 hops, so the average path length is $10/7$ and $\theta \le 7/10$. The random matrices get higher θ because their traffic is concentrated on fewer pairs, so more of it can go over direct links.